

DSA Complete Roadmap

25 Topics · 5 Phases · Continuous edition. Concept Questions, Coding Problems, and a Before-Moving-On checklist for every topic — covers service-based to top product-company interviews.

PHASE 1 — FOUNDATIONS

01

Complexity Analysis

Measure performance before writing a single line of code.

2-3 days | 1-3 hrs/day | Difficulty: Easy

WHAT IS IT?

Big-O notation describes how runtime or memory grows with input size n . Drop constants and lower-order terms - $O(2n)$ becomes $O(n)$. Always reason about the worst case. You will label every function you write from now on with a complexity, so this habit starts here.

KEY CONCEPTS

TERM	EXPLANATION
$O(1)$ - Constant	Array index or hash lookup. Time stays the same no matter how large the input gets.
$O(\log n)$ - Logarithmic	Binary search. Input is halved at every step, so time grows very slowly.
$O(n)$ - Linear	A single loop over the input. Time grows proportionally with size.
$O(n \log n)$	Merge sort, heap sort. The best achievable complexity for comparison-based sorting.
$O(n^2)$ - Quadratic	Two nested loops. Avoid this for n greater than roughly 10,000.
Space complexity	Extra memory used beyond the input. In-place algorithms use $O(1)$ extra space.
Best/worst/avg case	Quick sort is $O(n \log n)$ on average but $O(n^2)$ in the worst case.

CONCEPT QUESTIONS — answer in plain words

- Q1.** Access `arr[5]` in an array of 1000 elements - what is the time complexity and why?
- Q2.** Two nested loops each running n times - what is the overall complexity?
- Q3.** What does $O(\log n)$ mean intuitively? Why does binary search have this complexity?
- Q4.** Explain the difference between time complexity and space complexity.
- Q5.** For large n , which is faster - $O(n \log n)$ or $O(n^2)$? By roughly how much?

CODING PROBLEMS — solve on LeetCode

Write $O(1)/O(n)/O(n^2)$ functions — Label each and verify by reasoning about loop counts.

Write a loop halving n each step — Identify it as $O(\log n)$ and explain why.

Analyse any old code you wrote — Calculate its time and space complexity fully.

BEFORE MOVING ON

MUST — do not move on without these

- Explain $O(\log n)$ in plain English without looking it up.
- Calculate the complexity of any code with loops by inspection.
- Know why $O(n^2)$ is unacceptable for $n = 1,000,000$.

SOFT — good to have, not blocking

- Explain best vs worst vs average case with a sorting example.

RESOURCE: *Striver - Time Complexity video (YouTube)*

02

C++ STL Essentials

Know your tools. STL lets you focus on the problem, not the plumbing.

2-3 days | 1-3 hrs/day | Difficulty: Easy

WHAT IS IT?

The STL gives you vector, pair, unordered_map, map, set, stack, queue, priority_queue, and sort(). Know the time complexity of every operation cold. Use `m.count(key)` instead of `m[key]` to check existence, since `m[key]` silently inserts a default value.

KEY CONCEPTS

TERM	EXPLANATION
<code>vector<T></code>	Dynamic array. <code>push_back</code> is $O(1)$ amortised. Random access is $O(1)$.
<code>pair<A,B></code>	Use <code>make_pair(a,b)</code> or <code>{a,b}</code> . Access with <code>.first</code> and <code>.second</code> .
<code>unordered_map<K,V></code>	Hash map. Insert, find, erase are $O(1)$ average, $O(n)$ worst case.
<code>map<K,V></code>	Sorted balanced tree. All operations $O(\log n)$. Use when order matters.
<code>unordered_set<T></code>	Stores unique elements. $O(1)$ average lookup. No ordering guarantee.
<code>stack<T></code>	LIFO: <code>push()</code> , <code>pop()</code> , <code>top()</code> . Used for DFS, expression parsing, backtracking.
<code>queue<T></code>	FIFO: <code>push()</code> , <code>pop()</code> , <code>front()</code> . Used for BFS traversal.
<code>priority_queue<T></code>	Max-heap by default. For a min-heap use <code>priority_queue<int,vector<int>,greater<int>></code> .
<code>sort(begin,end)</code>	$O(n \log n)$. Custom order with a lambda: <code>sort(v.begin(),v.end(),[](a,b){...})</code> .

CONCEPT QUESTIONS — answer in plain words

Q1. What is the difference between map and unordered_map? When would you use each?

Q2. How do you create a min-heap with priority_queue? Write the declaration.

Q3. What does m[key] do when the key does not exist in the map?

Q4. Time complexity of inserting an element at the beginning of a vector?

Q5. How do you iterate over all key-value pairs in an unordered_map?

CODING PROBLEMS — solve on LeetCode

Count char frequency — Use unordered_map to count each character in a string.

Find top-3 elements — Use a min-heap priority_queue of size 3.

Remove duplicates — Copy a vector into an unordered_set and back. Observe the result.

BEFORE MOVING ON

MUST — do not move on without these

- Write unordered_map insert, lookup, and count from memory.
- Know the complexity of every STL container used in DSA.
- Create a min-heap priority_queue without looking it up.

SOFT — good to have, not blocking

- Write a custom comparator lambda for sort().

RESOURCE: Striver - C++ STL video (YouTube)

03

Recursion

Master the call stack. DP, backtracking, and trees all build on this.
3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

A function calling itself with a smaller input. It must have a base case or you get a stack overflow. Draw the call tree on paper for every recursive function before coding it. Return values bubble back up the stack from the base case to the root call.

KEY CONCEPTS

TERM	EXPLANATION
Base case	The stopping condition, e.g. factorial(0)=1. Missing it causes infinite recursion.
Recursive case	Calls itself with a smaller input, e.g. factorial(n)=n*factorial(n-1).
Call stack	Each call is one stack frame in memory, holding its own local variables.
Call tree	Draw it on paper first, always. Most recursion bugs come from skipping this.
Return values	Bubble back up the stack. factorial(3) waits for factorial(2) to return.
Multiple branches	Fibonacci calls itself twice per call, making the call tree exponential.
Tail recursion	The recursive call is the last thing done, which some compilers optimise.

CONCEPT QUESTIONS — answer in plain words

- Q1. What is a base case and what happens if you forget to write one?
- Q2. Draw the full call stack for factorial(4) - write the return value at each step.
- Q3. What is the time complexity of naive recursive Fibonacci? Why is it so bad?
- Q4. How does the call stack differ between factorial and Fibonacci?
- Q5. How would you convert a recursive function to iterative - what is the general approach?

CODING PROBLEMS — solve on LeetCode

Factorial — Trace the full call stack on paper before writing a single line of code.

Fibonacci — Run for $n=40$. Notice it freezes. Understand exactly why.

Print 1 to n — Recursively only - no loops allowed anywhere in the function.

BEFORE MOVING ON

MUST — do not move on without these

- Trace any recursive function on paper without running it.
- Identify the base case and recursive case in any given function.
- Know why forgetting the base case causes infinite recursion.

SOFT — good to have, not blocking

- Convert a simple recursive function to iterative using a stack.

RESOURCE: *Apna College - Recursion playlist, first 5 videos (YouTube)*

04

Arrays

Contiguous memory enables $O(1)$ access. Every other DS uses arrays underneath.

4-5 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Elements are stored back-to-back in memory. The address of `arr[i]` equals base + i times element size, computed directly in $O(1)$. Insert or delete in the middle costs $O(n)$ due to shifting. Master prefix sum, Kadane's algorithm, the Dutch National Flag, and two-pointer techniques on sorted arrays.

KEY CONCEPTS

TERM	EXPLANATION
Contiguous memory	<code>arr[i]</code> is found directly via the address formula - no traversal needed.
Static vs vector	Plain arrays are fixed size at compile time. <code>vector<int></code> resizes dynamically.
2D arrays	Stored row by row. <code>arr[i][j]</code> sits at offset $i \cdot \text{cols} + j$ in memory.
Prefix sum	$\text{preSum}[i] = \text{sum}(\text{arr}[0..i])$. Range sum $[l, r] = \text{preSum}[r] - \text{preSum}[l-1]$ in $O(1)$.
Kadane's algorithm	Track <code>maxEndingHere</code> and <code>maxSoFar</code> to find the maximum subarray sum in $O(n)$.

TERM	EXPLANATION
Dutch National Flag	Three pointers low, mid, high sort an array of 0s, 1s, and 2s in one pass.
Two pointer	Left and right indices converge inward on a sorted array to find pairs.

CONCEPT QUESTIONS — answer in plain words

- Q1.** Why is array element access $O(1)$ even for arrays with a billion elements?
- Q2.** Build the prefix sum for [3,1,4,1,5]. Find the sum of index 1 to 3 in $O(1)$.
- Q3.** Explain Kadane's algorithm in plain English. Trace it on [-2,1,-3,4,-1,2,1].
- Q4.** Given a sorted array and a target sum, how do you find a pair in $O(n)$?
- Q5.** An array of $n+1$ integers with values 1 to n - find the duplicate in $O(1)$ space.

CODING PROBLEMS — solve on LeetCode

LC #1 Two Sum — Hashmap approach, $O(n)$ time. Store the complement as you iterate.

LC #53 Max Subarray — Implement Kadane's algorithm from memory.

LC #724 Pivot Index — Prefix sum from both sides. $O(n)$ time, $O(1)$ space.

BEFORE MOVING ON

MUST — do not move on without these

- Implement prefix sum and use it for range queries.
- Code Kadane's algorithm from memory in under 5 minutes.
- Solve at least LC #1, #53, and #121 without hints.

SOFT — good to have, not blocking

- Implement the Dutch National Flag without looking at the solution.

RESOURCE: Striver Arrays + Apna College Arrays (YouTube)

05

Strings

Characters are integers. ASCII arithmetic is your secret weapon.
3-4 days | 1-3 hrs/day | Difficulty: Easy

WHAT IS IT?

Strings are character arrays. Characters are integers under the hood - 'a' is 97, 'A' is 65, '0' is 48. Memorise these. c-'a' maps a lowercase letter to a 0-25 index. Most string problems reduce to frequency counting, two pointer, or sliding window.

KEY CONCEPTS

TERM	EXPLANATION
ASCII values	'a'=97, 'A'=65, '0'=48. c-'a' gives a 0-25 index for lowercase letters.

TERM	EXPLANATION
Key methods	length(), substr(pos,len), find(sub), and more - know these cold.
Char functions	isalpha(), isdigit(), isspace(), tolower(), toupper() from <cctype>.
Frequency array	int freq[26]={0}; freq[c-'a']++ gives O(1) space character counting.
Palindrome check	Two pointers from both ends, moving inward, skipping non-alphanumeric chars.
Anagram check	Sort both strings in O(n log n), or compare frequency arrays in O(n).
String <-> int	stoi(str) converts to int. to_string(n) converts back to a string.

CONCEPT QUESTIONS — answer in plain words

- Q1. ASCII of 'a', 'A', '0' - write them from memory. Why do they matter in code?
- Q2. How do you check two strings are anagrams in O(n) time and O(1) space?
- Q3. Time complexity of concatenating with + in a loop of n iterations?
- Q4. Check a palindrome ignoring spaces and case - walk through your approach.
- Q5. Find the first non-repeating character - what data structure helps and why?

CODING PROBLEMS — solve on LeetCode

- LC #125 Valid Palindrome — Skip non-alphanumeric with isalnum(). tolower() before comparing.
- LC #242 Valid Anagram — Frequency array of size 26. Increment for s, decrement for t.
- LC #387 First Unique Char — Two-pass: build a frequency map, then find the first char with count 1.

BEFORE MOVING ON

MUST — do not move on without these

- Write the c-'a' char-to-index conversion from memory.
- Solve Valid Anagram in O(n) without sorting.
- Solve at least LC #125, #242, and #344 without hints.

SOFT — good to have, not blocking

- Implement length, substr, and find manually without built-ins.

RESOURCE: *Apna College Strings playlist (YouTube)*

06

Linked List

Non-contiguous nodes. Pointers are everything. Draw every step before coding.
4-5 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Nodes are scattered across memory, each holding data plus a pointer to the next node. There is no random access - you must traverse from the head in O(n). Head insertion is O(1). The three-pointer reversal and Floyd's cycle detection are the two must-know algorithms here.

KEY CONCEPTS

TERM	EXPLANATION
Node structure	struct Node{int data; Node* next;}. Head is the entry point of the list.
Traversal	Node* c=head; while(c){visit(c); c=c->next;} - the basic pattern for every problem.
Head insert	O(1): newNode->next=head; head=newNode. Compare to O(n) for arrays.
3-pointer reversal	prev=null, curr=head. Each step: save next, reverse curr's pointer, advance.
Floyd's algorithm	Slow moves 1 step, fast moves 2. If they ever meet, a cycle exists.
Find middle	Slow/fast pointer. When fast reaches the end, slow sits at the middle.
Merge sorted lists	Compare heads, attach the smaller node, advance that list's pointer.

CONCEPT QUESTIONS — answer in plain words

- Q1. Why is head insertion O(1) in a linked list but O(n) in an array?
- Q2. Explain Floyd's cycle detection - why do fast and slow pointers always meet?
- Q3. Walk through 3-pointer reversal for 1->2->3->4->null. Draw each step.
- Q4. How do you find the middle of a linked list in a single pass?
- Q5. When would you prefer a linked list over an array in a real program?

CODING PROBLEMS — solve on LeetCode

LC #206 Reverse Linked List — Draw the pointer state on paper fully before writing any code.

LC #876 Middle of LL — Slow/fast pointer. When fast or fast->next is null, slow is the middle.

LC #141 Linked List Cycle — Floyd's algorithm - if fast ever equals slow, a cycle exists.

BEFORE MOVING ON

MUST — do not move on without these

- Draw 3-pointer reversal step by step from memory.
- Explain Floyd's algorithm and why it works in plain words.
- Solve Reverse LL and Middle LL without any hints.

SOFT — good to have, not blocking

- Solve Palindrome LL in O(1) space without looking at the solution.

RESOURCE: *Apna College Linked List playlist (YouTube)*

07

Stack & Queue

LIFO and FIFO. Together they underpin BFS, DFS, and expression parsing.

3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Stack is Last In First Out. Queue is First In First Out. Both define a behaviour, not an implementation. Monotonic stack and min-stack are the two key interview patterns to master. Queues power BFS traversal across trees and graphs.

KEY CONCEPTS

TERM	EXPLANATION
Stack - LIFO	push() to top, pop() from top. Browser back button is a real-world example.
Queue - FIFO	push() to back, pop() from front. Powers BFS and task scheduling.
Implement from array	Stack: array plus a top index. Queue: circular buffer or two indices.
Queue from 2 stacks	A push-stack and a pop-stack. Transfer all elements when pop-stack is empty.
Monotonic stack	Stack values stay increasing or decreasing - used for next-greater-element problems.
Min Stack	Store (value, current_min) pairs so getMin() is O(1) at every state.
Balanced brackets	Push opening brackets. On a closing bracket, pop and check it matches.

CONCEPT QUESTIONS — answer in plain words

- Q1.** Give a real-world example of a stack and a queue - not textbook examples.
- Q2.** How do you implement a queue using two stacks? Explain push and pop.
- Q3.** What is a monotonic stack? Trace it on [4,5,2,10,8].
- Q4.** How does Min Stack achieve O(1) getMin? What is stored at each push?
- Q5.** Walk through the balanced parentheses check for the input ({[]}).

CODING PROBLEMS — solve on LeetCode

- LC #20 Valid Parentheses** — Push open brackets. On close, pop and verify the match.
- LC #155 Min Stack** — Store the pair (value, current_min) at each push operation.
- LC #232 Queue via Stacks** — Two stacks. Transfer everything when the pop-stack is empty.

BEFORE MOVING ON

MUST — do not move on without these

- Implement a queue from two stacks and explain amortised O(1).
- Trace the monotonic stack algorithm on a new array without help.
- Solve Valid Parentheses and Min Stack without hints.

SOFT — good to have, not blocking

- Implement stack and queue from scratch using arrays only.

RESOURCE: *Striver Stack & Queue series (YouTube)*

08

Hashing & Hashmaps

$O(1)$ lookup converts $O(n^2)$ brute force into $O(n)$ elegance.

3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

A hash map converts a key into an array index via a hash function, enabling $O(1)$ average insert, lookup, and delete. The core pattern is: store what you have seen, then look it up instead of searching. Two Sum is the archetype - store complements while iterating instead of nesting loops.

KEY CONCEPTS

TERM	EXPLANATION
Hash function	Maps a key to an array index. A good hash spreads keys evenly to avoid collisions.
Collision handling	Chaining stores a linked list per slot. Open addressing probes for the next free slot.
$O(1)$ avg, $O(n)$ worst	Worst case happens when all keys hash to the same slot.
unordered_map ops	$m[k]=v$, $m[k]$, $m.count(k)$, $m.find(k)$, $m.erase(k)$ - all $O(1)$ average.
map vs unordered_map	map is sorted, $O(\log n)$. unordered_map is unsorted, $O(1)$ average.
Frequency count	<code>for(int x:arr) count[x]++</code> counts occurrences of every element in $O(n)$.
Two Sum pattern	for each i : if <code>map.count(target-arr[i])</code> then found; else store <code>arr[i]</code> .

CONCEPT QUESTIONS — answer in plain words

- Q1. What is a hash function and what makes a good one?
- Q2. Why is unordered_map $O(1)$ average but $O(n)$ worst case?
- Q3. What does `m[key]` do if the key does not exist? Why is this dangerous?
- Q4. Explain the Two Sum hashmap approach - why $O(n)$ instead of $O(n^2)$?
- Q5. When would you choose map over unordered_map? Give a concrete example.

CODING PROBLEMS — solve on LeetCode

LC #1 Two Sum — Single pass hashmap. Store the complement as value, index as key.

LC #217 Contains Duplicate — Insert into a set. Return true if insert finds it already present.

LC #290 Word Pattern — Two maps - char to word and word to char. Both must agree.

BEFORE MOVING ON

MUST — do not move on without these

- Implement the Two Sum hashmap approach from memory in 5 minutes.
- Explain why `m[key]` for an existence check is dangerous.
- Solve Two Sum, Contains Duplicate, and Word Pattern without hints.

SOFT — good to have, not blocking

- Solve Group Anagrams - the Medium stretch goal.

RESOURCE: *Striver Hashing series (YouTube)*

09

Searching

Binary search on sorted data. Binary search on answer space. Both $O(\log n)$.
3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Linear search is $O(n)$ and works on unsorted data. Binary search is $O(\log n)$ and needs sorted or monotonic data. Key formula: $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ to avoid integer overflow. Binary search also applies to answer spaces - any monotonic function where you search over a range of possible answers.

KEY CONCEPTS

TERM	EXPLANATION
Linear search	Check each element in order. $O(n)$ time, $O(1)$ space.
Binary search idea	If <code>arr[mid]==target</code> , found. If <code>arr[mid]<target</code> , search the right half, else left.
Safe mid formula	$\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ avoids the overflow that $(\text{lo} + \text{hi}) / 2$ can cause.
Loop condition	$\text{lo} \leq \text{hi}$ for a standard search; $\text{lo} < \text{hi}$ for boundary-finding variants.
Lower/upper bound	<code>lower_bound</code> finds the first index where <code>arr[i] >= target</code> .
Rotated sorted array	Determine which half is sorted, then check if the target lies within it.
BS on answer space	'Minimise the maximum' problems binary search over the range of possible answers.

CONCEPT QUESTIONS — answer in plain words

- Q1. Why does binary search require sorted data? What breaks on unsorted input?
- Q2. Why use $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ instead of $(\text{lo} + \text{hi}) / 2$? Give a concrete overflow example.
- Q3. What is the loop invariant of binary search after every iteration?
- Q4. Describe 'binary search on answer space' - give a concrete problem example.
- Q5. What do `lower_bound` and `upper_bound` return when the target is not in the array?

CODING PROBLEMS — solve on LeetCode

LC #704 Binary Search — Iterative. Return the index if found, -1 if not. Use the $\text{lo} \leq \text{hi}$ condition.

LC #35 Search Insert Position — When the target is not found, `lo` is the answer. Explain why.

LC #278 First Bad Version — If mid is bad, the answer is in the left half including mid.

BEFORE MOVING ON

MUST — do not move on without these

- Write iterative binary search from memory with correct lo/hi/mid.
- Explain why lo is the insert position when the target is not found.
- Solve LC #704, #35, and #278 without hints.

SOFT — good to have, not blocking

- Handle the rotated sorted array variant without looking at hints.

RESOURCE: *Striver Binary Search series (YouTube)*

10

Sorting Algorithms

*Understand sorting mechanics before calling sort() on everything.
4-5 days | 1-3 hrs/day | Difficulty: Medium*

WHAT IS IT?

You must be able to implement sorting from scratch - interviewers ask, and the underlying ideas appear elsewhere, like merge sort's divide-and-conquer showing up in counting inversions. In practice, use C++'s built-in `sort()`, which is an $O(n \log n)$ introsort hybrid.

KEY CONCEPTS

TERM	EXPLANATION
Bubble sort $O(n^2)$	Swap adjacent elements if out of order. Repeat until no swaps happen.
Selection sort $O(n^2)$	Find the minimum in the remaining unsorted part. Swap it to the front.
Insertion sort $O(n^2)$	Build the sorted portion left to right. $O(n)$ on nearly sorted input.
Merge sort	Divide in half recursively. Merge sorted halves in $O(n)$. $O(n \log n)$ overall.
Quick sort	Pick a pivot. Partition around it. $O(n \log n)$ average, $O(n^2)$ worst case.
Stability	A stable sort preserves the relative order of equal elements.
C++ sort()	Introsort hybrid. $O(n \log n)$ guaranteed, not stable by default.

CONCEPT QUESTIONS — answer in plain words

- Q1. Explain merge sort - what is the base case and how does the merge step work?
- Q2. Why is quick sort $O(n^2)$ worst case? When exactly does this happen?
- Q3. What does a stable sort mean? Give an example where stability matters.
- Q4. When would you prefer insertion sort over merge sort despite worse complexity?
- Q5. Time and space complexity of merge sort vs quick sort - what is the key difference?

CODING PROBLEMS — solve on LeetCode

Implement merge sort — Do not look at a reference. The merge step is where most bugs occur.

Implement quick sort — Random pivot. Partition: smaller left, larger right. Recurse both sides.

LC #75 Sort Colors — Dutch National Flag. $O(n)$ single pass with three pointers.

BEFORE MOVING ON

MUST — do not move on without these

- Implement merge sort from scratch without a reference.
- Explain why merge sort is $O(n \log n)$ - trace the recurrence.
- Know why quick sort degrades to $O(n^2)$ on sorted input.

SOFT — good to have, not blocking

- Implement quick sort with partitioning that handles duplicates correctly.

RESOURCE: *Apna College Sorting playlist (YouTube)*

11

Bit Manipulation *FEATURED*

Every integer is bits. Bitwise ops run $O(1)$ and appear in Amazon/Google rounds.
2-3 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Bitwise operators work directly on binary representations. AND clears bits, OR sets bits, XOR toggles bits and finds unique elements, and shifts multiply or divide by powers of two. It's a small topic but comes up frequently at product-company interviews.

KEY CONCEPTS

TERM	EXPLANATION
AND $a \& b$	1 if both bits are 1. Clear bit i : $a \& \sim(1 \ll i)$.
OR $a b$	1 if either bit is 1. Set bit i : $a (1 \ll i)$.
XOR $a \wedge b$	1 if bits differ. Toggle bit i : $a \wedge (1 \ll i)$. Pairs of equal numbers cancel to 0.
Left shift $a \ll k$	Multiplies by 2^k . $a \ll 1$ equals $a * 2$; $a \ll 3$ equals $a * 8$.
Right shift $a \gg k$	Divides by 2^k . Arithmetic shift preserves the sign for negative numbers.
Check bit i	$(a \gg i) \& 1$ returns 1 if bit i is set, 0 otherwise.
Power of 2 check	$n > 0 \&\& (n \& (n - 1)) == 0$ - powers of 2 have exactly one set bit.
Count set bits	<code>__builtin_popcount(n)</code> in GCC, or loop <code>while(n){n&=(n-1); count++;}</code> .
XOR unique element	XOR all elements together. Duplicate pairs cancel, leaving the unique one.

CONCEPT QUESTIONS — answer in plain words

- Q1. What does XOR of a number with itself equal? How does this help find the unique element?
- Q2. How do you check if a number is a power of 2 using a single bitwise operation?
- Q3. What is the result of left-shifting 1 by k positions? Express it as a formula.
- Q4. Why does & have lower precedence than ==? What does this mean practically?
- Q5. What is `__builtin_popcount` and when would you use it in an interview?

CODING PROBLEMS — solve on LeetCode

LC #136 Single Number — XOR all elements. Pairs cancel. The unique number remains.

LC #191 Number of 1 Bits — Loop: count $n \& 1$ then right shift. Or use `__builtin_popcount`.

LC #231 Power of Two — Single check: $n > 0 \ \&\& \ (n \& (n-1)) == 0$.

BEFORE MOVING ON

MUST — do not move on without these

- Write bit check, set, clear, and toggle for bit i from memory.
- Explain why XOR finds the unique element in an array of pairs.
- Solve Single Number and Power of Two without hints.

SOFT — good to have, not blocking

- Solve Sum of Two Integers without using + or - operators.

RESOURCE: [NeetCode Bit Manipulation playlist](#) (6 short videos, one sitting)

PHASE 2 — PATTERNS

12

Two Pointers

Replace $O(n^2)$ nested loops with $O(n)$ two-index elegance.
3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Maintain left and right indices that move toward each other or in the same direction. This eliminates nested loops by making smart decisions about which pointer to move next. The classic pair-sum variant requires a sorted array.

KEY CONCEPTS

TERM	EXPLANATION
Opposite direction	Start at both ends. Move inward based on a comparison with the target.
Same direction	One pointer writes, the other reads - used to remove duplicates in place.
Sorted prerequisite	The pair-sum two-pointer technique only works on a sorted array.
Move decision	Sum greater than target: move right pointer left. Sum less: move left pointer right.

TERM	EXPLANATION
Three sum	Fix one element, then run two-pointer on the remaining sorted array.
Remove duplicates	A slow pointer marks the write position; a fast pointer scans ahead.
Container with water	Height is limited by the shorter wall - always move the shorter one inward.

CONCEPT QUESTIONS — answer in plain words

- Q1. Why must the array be sorted for the opposite-direction two-pointer to work?
- Q2. Explain why you move the shorter wall in Container With Water - prove it.
- Q3. How does the same-direction two-pointer remove duplicates in-place?
- Q4. Why is two-pointer $O(n)$ while brute-force pair-sum is $O(n^2)$?
- Q5. Describe 3Sum - how does fixing one element reduce it to a Two Sum problem?

CODING PROBLEMS — solve on LeetCode

- LC #167 Two Sum II — Sorted input. left=0, right=n-1. Move based on sum vs target.
- LC #11 Container With Water — Move the shorter wall inward at each step.
- LC #15 3Sum — Sort first. Fix one element. Two-pointer on the rest, skipping duplicates.

BEFORE MOVING ON

MUST — do not move on without these

- Explain why you move the shorter pointer in Container With Water.
- Implement the opposite-direction two-pointer pair-sum from memory.
- Solve Two Sum II and Remove Duplicates without hints.

SOFT — good to have, not blocking

- Solve 3Sum including all three duplicate-skipping edge cases.

RESOURCE: *Striver Two Pointers series (YouTube)*

13

Sliding Window

Maintain a window. Slide it. Never recompute from scratch.
3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

A contiguous subarray window slides across the array, updating in $O(1)$ by adding the incoming element and removing the outgoing one. A fixed window has a constant size k ; a variable window expands and shrinks based on a constraint. Recomputing from scratch each time defeats the purpose of this pattern.

KEY CONCEPTS

TERM	EXPLANATION
Fixed window	Size k stays constant. Slide: $\text{sum} += \text{arr}[r]$; $\text{sum} -= \text{arr}[r-k]$; $r++$.
Variable window	Expand right. When the constraint is violated, shrink from the left.
$O(1)$ update	$\text{sum} += \text{arr}[\text{right}]$ then $\text{sum} -= \text{arr}[\text{left}++]$ avoids recomputing the whole window.
Max in window	A deque holding indices in decreasing value order gives the max in $O(1)$.
String window	Combine a character frequency map with a sliding window over the string.
Longest substring	Track a set of seen characters. Expand right, shrink left on a duplicate.
Minimum window	Expand until all required characters are present, then shrink to minimise length.

CONCEPT QUESTIONS — answer in plain words

- Q1.** What is the key insight that makes sliding window $O(n)$ instead of $O(nk)$?
- Q2.** Difference between fixed-size and variable-size sliding window - example of each.
- Q3.** When does the variable window shrink from the left? Can it shrink multiple times?
- Q4.** How do you track the maximum in a sliding window of size k efficiently?
- Q5.** Describe the 'expand right, shrink left' pattern for longest substring problems.

CODING PROBLEMS — solve on LeetCode

LC #643 Max Average Subarray — First compute the window sum, then slide: add new, subtract old.

LC #3 Longest No Repeat — Set for seen chars. Shrink left on duplicate. Track the max length.

LC #209 Min Size Subarray — Expand until $\text{sum} \geq \text{target}$. Shrink to minimise length. Track the min.

BEFORE MOVING ON

MUST — do not move on without these

- Explain in one sentence how $O(nk)$ becomes $O(n)$ with sliding window.
- Implement a fixed sliding window sum update from memory.
- Solve Max Average Subarray and Longest Substring Without Repeat.

SOFT — good to have, not blocking

- Solve Minimum Size Subarray Sum with correct shrinking logic.

RESOURCE: Striver Sliding Window series (YouTube)

14

Binary Trees

Hierarchical data. Master the four traversals - everything else builds on them.
5-6 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

A binary tree has at most two children per node. Every tree algorithm reduces to a traversal plus some logic at each node. Always handle the null node as the base case first. Inorder traversal of a BST gives sorted output.

KEY CONCEPTS

TERM	EXPLANATION
Node structure	struct Node{int val; Node *left,*right;}. Null marks the absence of a child.
Inorder (L, Root, R)	Left, visit, right. Gives sorted output for a binary search tree.
Preorder (Root, L, R)	Visit the node first. Used for copying trees and prefix expressions.
Postorder (L, R, Root)	Children before the node. Used for safe deletion of the tree.
Level order (BFS)	Use a queue. Process one level at a time by tracking queue.size().
Height	$\max(\text{height}(\text{left}), \text{height}(\text{right})) + 1$. Null node returns 0.
Diameter	At each node, $\text{left_height} + \text{right_height}$ is a candidate for the answer.

CONCEPT QUESTIONS — answer in plain words

- Q1.** What are the four traversals and what output order do they produce?
- Q2.** How do you compute height recursively? Write the recurrence and base case.
- Q3.** What is the diameter and why doesn't it always pass through the root?
- Q4.** How do you do level-order traversal? How do you separate output by level?
- Q5.** What is the LCA of two nodes? Describe the recursive approach.

CODING PROBLEMS — solve on LeetCode

- LC #104 Max Depth** — $\max(\text{depth}(\text{left}), \text{depth}(\text{right})) + 1$. Null returns 0.
- LC #543 Diameter** — At each node track $\text{left_h} + \text{right_h}$. Return height upward.
- LC #102 Level Order** — Queue. Process queue.size() nodes per level. Collect into rows.

BEFORE MOVING ON

MUST — do not move on without these

- Write all four traversals from memory without a reference.
- Compute tree height and diameter with correct base cases.
- Solve Max Depth, Invert Tree, and Diameter without hints.

SOFT — good to have, not blocking

- Implement level-order traversal that separates each level into its own list.

RESOURCE: [Apna College Binary Trees + Striver Tree series \(YouTube\)](#)

15

Binary Search Tree

Sorted binary tree. One invariant - $left < node < right$ - enables everything.

4-5 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

The BST invariant states that for every node, all left subtree values are smaller and all right subtree values are larger. This enables $O(\log n)$ average search, though it degrades to $O(n)$ if unbalanced. Inorder traversal gives sorted output - exploit this for most BST problems.

KEY CONCEPTS

TERM	EXPLANATION
BST invariant	$left < node < right$ must hold for the entire subtree, not just direct children.
Search $O(\log n)$	Compare with the root. Go left if smaller, right if larger.
Insert	Search for the correct position, then insert as a new leaf node.
Delete - 3 cases	Leaf: remove directly. One child: bypass it. Two children: use inorder successor.
Validate BST	Pass min/max bounds down the recursion, not just a comparison with children.
Kth smallest	Inorder gives sorted output - stop at the kth element visited.
Inorder successor	The smallest node greater than a given node. Found by going right once, then left.

CONCEPT QUESTIONS — answer in plain words

- Q1.** State the BST invariant precisely - for every node, what must hold?
- Q2.** Why does checking only $node > left$ child fail? Draw a counterexample.
- Q3.** Walk through deletion of a node with two children - what is the inorder successor?
- Q4.** Why does inorder traversal of a BST give sorted output?
- Q5.** What is the average and worst-case search time? When does the worst case occur?

CODING PROBLEMS — solve on LeetCode

- LC #700 Search BST** — Compare with root, go left or right. Return null if not found.
- LC #98 Validate BST** — Pass (min,max) bounds. Left subtree: $max = node.val$. Right: $min = node.val$.
- LC #230 Kth Smallest** — Inorder traversal. Stop at the kth element using a counter or list.

BEFORE MOVING ON

MUST — do not move on without these

- Draw a tree that looks valid but violates the BST invariant.
- Implement BST search and insert from memory.
- Solve Search BST and LCA of BST without hints.

SOFT — good to have, not blocking

- Implement Validate BST with the min/max bounds approach.

RESOURCE: *Striver BST series (YouTube)*

16

Heaps & Priority Queue

Always get min or max in $O(\log n)$. Essential for top-K and greedy problems.
3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

A heap is a complete binary tree stored in an array. The parent is always $\geq$ children in a max-heap, or $\leq$ in a min-heap. The root gives the min or max in $O(1)$. Insert and delete are both $O(\log n)$. STL `priority_queue` is a max-heap by default, and heaps power Dijkstra, top-K, and merge-K patterns.

KEY CONCEPTS

TERM	EXPLANATION
Heap property	Max-heap: $\text{parent} \geq \text{children}$. Min-heap: $\text{parent} \leq \text{children}$, at every level.
Array storage	Node at index i : left child at $2i+1$, right at $2i+2$, parent at $(i-1)/2$.
Heapify up	After insert: swap the new element with its parent while it violates the heap property.
Heapify down	After removing the root: move the last element to the root, sink it down.
Build heap $O(n)$	Heapify down starting from the last internal node - not $O(n \log n)$.
Top-K pattern	Keep a min-heap of size k . Push each element, pop if the size exceeds k .
Merge K sorted	Min-heap seeded with the head of each list. Pop min, push its successor.

CONCEPT QUESTIONS — answer in plain words

- Q1. What property does a max-heap maintain? How is it stored in an array?
- Q2. For a node at index i , what are the indices of its left child, right child, and parent?
- Q3. For the top-K largest elements, do you use a min-heap or max-heap? Why counterintuitive?
- Q4. Time complexity of building a heap from n elements - is it $O(n)$ or $O(n \log n)$?
- Q5. How do you merge K sorted lists using a heap? What do you store in the heap?

CODING PROBLEMS — solve on LeetCode

LC #703 Kth Largest Stream — Min-heap of size k . The top is always the k th largest.

LC #215 Kth Largest in Array — Min-heap of size k . Push each element, pop if size exceeds k .

LC #347 Top K Frequent — Count frequencies. Min-heap of (freq, element) pairs, size k.

BEFORE MOVING ON

MUST — do not move on without these

- State the array index formulas for parent and children from memory.
- Explain why a min-heap gives you the K largest elements.
- Solve Kth Largest Stream and Top K Frequent without hints.

SOFT — good to have, not blocking

- Implement Merge K Sorted Lists using a heap.

RESOURCE: Striver Heaps series (YouTube)

17

Math & Number Theory *FEATURED*

GCD, primes, and modular arithmetic unlock a surprising number of problems.
3-4 days | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

GCD and LCM show up in fraction and interval problems. The Sieve of Eratosthenes finds all primes up to n in $O(n \log \log n)$. Modular arithmetic prevents overflow in DP counting problems. Fast exponentiation computes $a^n \bmod m$ in $O(\log n)$. This fits before graphs because graph problems frequently need modular arithmetic too.

KEY CONCEPTS

TERM	EXPLANATION
GCD	Euclidean algorithm: $\text{gcd}(a,b)=\text{gcd}(b,a\%b)$. Base case: $\text{gcd}(a,0)=a$.
LCM formula	$\text{lcm}(a,b) = (a/\text{gcd}(a,b))*b$. Divide first to avoid overflow.
Prime check $O(\sqrt{n})$	Test divisibility from 2 to $\sqrt{n}$. If none divide it, it is prime.
Sieve of Eratosthenes	Mark multiples of each prime as composite. Unmarked numbers are prime.
Modular arithmetic	$(a+b)\%m = ((a\%m)+(b\%m))\%m$. Apply this rule at every intermediate step.
Why mod $1e9+7$	$1e9+7$ is prime and fits in a 32-bit int - the standard modulus for DP counting.
Fast exponentiation	$a^n = (a^{(n/2)})^2$ if n is even. Runs in $O(\log n)$ instead of $O(n)$.
Modular inverse	$a^{-1} \bmod p = a^{(p-2)} \bmod p$, valid when p is prime (Fermat's little theorem).

CONCEPT QUESTIONS — answer in plain words

- Q1. How does the Euclidean algorithm compute GCD? Trace $\text{gcd}(48,18)$ step by step.
- Q2. Why compute LCM as $(a/\text{gcd})*b$ instead of $(a*b)/\text{gcd}$?
- Q3. How does the Sieve of Eratosthenes work? What is its time complexity?
- Q4. Why is modular arithmetic needed in DP? When does overflow happen without it?

Q5. What is modular inverse? When do you need it and how do you compute it?

CODING PROBLEMS — solve on LeetCode

LC #204 Count Primes — Sieve of Eratosthenes. Mark composites, count what remains unmarked.

LC #1979 Find GCD of Array — Apply std::gcd iteratively across the array.

LC #50 Pow(x,n) — Fast exponentiation. Handle negative n. Runs in $O(\log n)$.

BEFORE MOVING ON

MUST — do not move on without these

- Implement gcd using the Euclidean algorithm from memory.
- Know why $(a*b)/gcd$ overflows and what the correct formula is.
- Implement the Sieve of Eratosthenes from scratch.

SOFT — good to have, not blocking

- Implement modular fast exponentiation for $a^n \bmod m$.

RESOURCE: NeetCode Math playlist + GFG Modular Arithmetic article

PHASE 3 — MORE PATTERNS

18

Graphs (BFS & DFS)

The most powerful structure. Maps, networks, and dependencies are all graphs.
2-3 weeks | 1-3 hrs/day | Difficulty: Hard

WHAT IS IT?

Nodes connected by edges - a graph can have cycles, disconnected parts, and directed or undirected edges. BFS uses a queue for level-by-level traversal and shortest path in unweighted graphs. DFS uses recursion for cycle detection, topological sort, and finding connected components. Always maintain a visited[] array or you will loop forever.

KEY CONCEPTS

TERM	EXPLANATION
Adjacency list	<code>vector<vector<int>> adj(n). adj[u].push_back(v)</code> for an edge u-v.
Adjacency matrix	<code>mat[u][v]=1</code> if an edge exists. $O(1)$ edge check but $O(V^2)$ space.
BFS with queue	Start node into the queue. Pop, visit unvisited neighbours, push them.
DFS recursion	Visit the node, mark it visited, recurse on all unvisited neighbours.
Cycle - undirected	DFS with parent tracking. A visited neighbour that isn't the parent means a cycle.
Cycle - directed	DFS with a recursion stack. A node visited within the current stack means a cycle.
Bipartite check	BFS with 2-colouring. Same-colour adjacent nodes mean it isn't bipartite.

TERM	EXPLANATION
Topological sort	DFS post-order reversed gives a valid topological order for a DAG.
Components	Run BFS or DFS from every unvisited node, counting how many times you start fresh.

CONCEPT QUESTIONS — answer in plain words

- Q1. Why does BFS/DFS without visited[] loop forever on a cyclic graph?
- Q2. What is the difference between BFS and DFS - when do you choose each?
- Q3. How do you detect a cycle in an undirected graph using DFS?
- Q4. What is topological sort? Why does it only work on DAGs?
- Q5. How do you find all connected components using BFS?

CODING PROBLEMS — solve on LeetCode

LC #200 Number of Islands — BFS/DFS from each unvisited '1'. Count how many times you start.

LC #207 Course Schedule — Cycle detection in a directed graph using DFS with a recursion stack.

LC #994 Rotting Oranges — Multi-source BFS. Start all rotten oranges in the queue simultaneously.

BEFORE MOVING ON

MUST — do not move on without these

- Implement BFS and DFS from scratch for an adjacency-list graph.
- Detect a cycle in both a directed and an undirected graph.
- Solve Number of Islands and Course Schedule without hints.

SOFT — good to have, not blocking

- Implement topological sort using both DFS and Kahn's algorithm.

RESOURCE: *Striver Graph Series* + *Apna College* - spend at least 2 weeks here

19

Backtracking

Systematic trial and error. Choose, explore, unchoose. Draw the tree first.
 1-1.5 weeks | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Build solutions incrementally. When a partial solution cannot lead to a valid complete one, undo the last choice and try the next option. The template is choose, explore, unchoose. Drawing the decision tree before coding is non-negotiable for this topic.

KEY CONCEPTS

TERM	EXPLANATION
The template	choose(opt); recurse(); unchoose(opt) - applies to nearly every backtracking problem.

TERM	EXPLANATION
Decision tree	Each level is a decision point; each branch is one available choice.
Pruning	Skip branches that can't lead to a valid solution, saving huge amounts of work.
Subsets (2^n)	At each element: include it or skip it. No sorting or duplicate handling needed.
Permutations ($n!$)	At each position, try each unused element via a <code>used[]</code> boolean array.
Combinations	Choose k from n by passing a start index to avoid repeating earlier elements.
N-Queens	Place one queen per row, checking column and both diagonals for conflicts.
Skip duplicates	Sort first, then skip a value if it matches the previous one at the same recursion level.

CONCEPT QUESTIONS — answer in plain words

- Q1.** What is the 'choose-explore-unchoose' template? Write it in pseudocode.
- Q2.** Why is the unchoose step as important as the choose step?
- Q3.** How do you avoid duplicate subsets when the input has duplicate values?
- Q4.** What is pruning in backtracking? Give an example where it helps significantly.
- Q5.** Draw the decision tree for subsets of `[1,2,3]`. How many leaf nodes are there?

CODING PROBLEMS — solve on LeetCode

- LC #78 Subsets** — At each element: include or skip. Two recursive branches.
- LC #46 Permutations** — `used[]` array. At each position try each unused element.
- LC #39 Combination Sum** — Sort. At each step pick the element or move to the next. Repeats allowed.

BEFORE MOVING ON

MUST — do not move on without these

- Write the backtracking template from memory in pseudocode.
- Explain why forgetting the unchoose step produces wrong results.
- Solve Subsets and Permutations without hints.

SOFT — good to have, not blocking

- Solve N-Queens - the hardest classic backtracking problem.

RESOURCE: Striver Backtracking series (YouTube)

WHAT IS IT?

A trie is a tree where each node represents one character. All strings sharing a prefix share the same path from the root. Insert and search are both $O(L)$, where L is the word length, independent of the total number of words stored - far faster than hashmaps for prefix queries like autocomplete.

KEY CONCEPTS

TERM	EXPLANATION
TrieNode structure	struct TrieNode{TrieNode* ch[26]; bool isEnd;}. Fixed array of 26 children.
Insert $O(L)$	For each character: create the child if missing, then move into it.
Search $O(L)$	For each character: if the child is missing, return false immediately.
startsWith $O(L)$	Same traversal as search, but returns true without checking isEnd.
children[26]	Indexed 0-25 via <code>ch[c-'a']</code> - a fixed array per node for lowercase letters.
isEnd flag	Marks that a complete word ends at this node, not just a prefix.
vs HashMap	Trie gives $O(L)$ prefix queries. HashMap gives $O(1)$ exact lookup but no prefix support.
Delete	Set <code>isEnd=false</code> , then prune any childless nodes left behind.

CONCEPT QUESTIONS — answer in plain words

- Q1.** How does a Trie differ from a hashmap for storing a dictionary of words?
- Q2.** Why does `search()` need `isEnd=true` but `startsWith()` does not?
- Q3.** Trace `insert('apple')` on an empty Trie - draw the node structure.
- Q4.** What is the time complexity of `search()` in a Trie with 1 million words?
- Q5.** How do you implement delete in a Trie? What makes it tricky?

CODING PROBLEMS — solve on LeetCode

LC #208 Implement Trie — Implement insert, search, and startsWith from scratch. Must-do.

LC #211 Add and Search — '.' matches any character - use DFS recursively through all children.

LC #648 Replace Words — Build a Trie from the roots. Find the shortest matching prefix for each word.

BEFORE MOVING ON

MUST — do not move on without these

- Write the TrieNode structure and insert/search from memory.
- Explain the difference between search and startsWith precisely.

- Implement LC #208 from scratch without any reference.

SOFT — good to have, not blocking

- Solve Word Search II using Trie plus DFS on the grid.

RESOURCE: Striver Tries series (YouTube)

PHASE 4 — ADVANCED ALGORITHMS

21

Dynamic Programming

DP = recursion + memory. If you can recurse it, you can DP it.

4-6 weeks | 1-3 hrs/day | Difficulty: Hard

WHAT IS IT?

Dynamic programming breaks a problem into overlapping subproblems and stores results to avoid recomputation. Memoization is the top-down version; tabulation is bottom-up. Every DP problem needs optimal substructure and overlapping subproblems. Expect 2-3 weeks before it truly clicks - that is completely normal.

KEY CONCEPTS

TERM	EXPLANATION
Optimal substructure	The solution is built from optimal solutions to its subproblems.
Overlapping subproblems	fib(5) calls fib(3) twice - store the result after computing it once.
Memoization	A recursive solution plus a memo array. Check the memo before recursing.
Tabulation	Fill a dp[] array from the base cases upward, with no recursion overhead.
State definition	What exactly does dp[i] mean? Getting this precise is most of the battle.
1D DP	Climbing stairs, house robber. dp[i] built from dp[i-1] and/or dp[i-2].
2D DP	Grid paths, LCS. dp[i][j] built from dp[i-1][j], dp[i][j-1], or dp[i-1][j-1].
0/1 Knapsack	Include or skip item i. $dp[i][w] = \max(dp[i-1][w], val[i] + dp[i-1][w - wt[i]])$.
LCS pattern	dp[i][j]: on a match, $dp[i-1][j-1] + 1$; otherwise $\max(dp[i-1][j], dp[i][j-1])$.

CONCEPT QUESTIONS — answer in plain words

- Q1.** What are the two properties a problem must have for DP to apply?
- Q2.** Explain the difference between memoization and tabulation - pros and cons of each.
- Q3.** What does dp[i] represent in the house robber problem? Write the recurrence.
- Q4.** Build the DP table for LCS of 'ABCBDAB' and 'BDCAB'.
- Q5.** How does 0/1 Knapsack differ from fractional Knapsack? Which one needs DP?

CODING PROBLEMS — solve on LeetCode

LC #70 Climbing Stairs — $dp[i] = dp[i-1] + dp[i-2]$. Identical to Fibonacci. $O(n)$ time, $O(1)$ space.

LC #322 Coin Change — $dp[amount] = \min \text{ coins}$. For each coin, update all reachable amounts.

LC #300 Longest Increasing Subsequence — $dp[i] = \max \text{ LIS ending at } i$. $O(n^2)$, or $O(n \log n)$ with patience sorting.

BEFORE MOVING ON

MUST — do not move on without these

- Solve Climbing Stairs and House Robber using tabulation.
- Explain what $dp[i]$ represents - the state definition - before coding.
- Solve Coin Change and at least one 2D DP problem.

SOFT — good to have, not blocking

- Implement 0/1 Knapsack from scratch in both memo and tabulation form.

RESOURCE: *Striver DP Series* - do the entire series in order, best-structured DP resource

22

Greedy Algorithms

Make locally optimal choices. Prove they lead to the global optimum.

1-1.5 weeks | 1-3 hrs/day | Difficulty: Medium

WHAT IS IT?

Greedy makes the best available choice at each step without reconsidering it later. Unlike DP, it never backtracks. The hard part is proving your greedy choice is correct - that the local optimum is genuinely part of the global optimum - rather than just assuming it.

KEY CONCEPTS

TERM	EXPLANATION
Greedy choice property	The local optimal choice is always part of some globally optimal solution.
No backtracking	Once committed to a choice, greedy never revisits or undoes it.
Activity selection	Sort by end time. Pick the earliest-ending activity that doesn't conflict.
Interval scheduling	Sort by start time. Merge intervals whose ranges overlap.
Fractional knapsack	Sort by value/weight ratio. Take the most valuable items first, fractions allowed.
Jump game	Track the max reachable index. If the current index exceeds it, return false.
When greedy fails	0/1 Knapsack has no fractions allowed, so greedy fails and DP is required.
Proof technique	Exchange argument - show that swapping the greedy choice never improves the result.

CONCEPT QUESTIONS — answer in plain words

Q1. What is the greedy choice property? How do you prove a greedy algorithm is correct?

- Q2. Why does sorting by end time work for activity selection? Prove it is optimal.
- Q3. Give an example where a greedy approach gives the wrong answer. Explain why.
- Q4. How does Jump Game work greedily? What does maxReach represent at each step?
- Q5. Why does greedy fail for 0/1 Knapsack but work for fractional Knapsack?

CODING PROBLEMS — solve on LeetCode

LC #55 Jump Game — Track max reachable. If $i > \text{maxReach}$ return false. Update maxReach each step.

LC #56 Merge Intervals — Sort by start. Merge if the current start $\leq$ previous end.

LC #435 Non-overlapping Intervals — Sort by end time. Count intervals to remove to eliminate overlaps.

BEFORE MOVING ON

MUST — do not move on without these

- Explain the greedy choice property and give a correct proof sketch.
- Identify at least one problem where greedy fails - explain why.
- Solve Jump Game and Merge Intervals without hints.

SOFT — good to have, not blocking

- Solve Gas Station - a tricky greedy problem with a circular constraint.

RESOURCE: *Striver Greedy series (YouTube)*

PHASE 5 — ADVANCED DATA STRUCTURES

23

Advanced Graphs

Weighted edges, shortest paths, spanning trees, and union-find.
2-3 weeks | 1-3 hrs/day | Difficulty: Hard

WHAT IS IT?

This extends BFS/DFS with weighted-graph algorithms. Dijkstra finds the shortest path in $O((V+E) \log V)$. Bellman-Ford handles negative weights in $O(VE)$. DSU (Disjoint Set Union) answers connectivity queries in near- $O(1)$. Kruskal and Prim build minimum spanning trees.

KEY CONCEPTS

TERM	EXPLANATION
Dijkstra	Min-heap of (dist,node). Relax edges greedily from the closest unvisited node.
Bellman-Ford	Relax all edges $V-1$ times. A V th relaxation that still improves reveals a negative cycle.
DSU find()	Finds the root of a set with path compression, making it amortised near- $O(1)$.
DSU union()	Merge by rank or size - attach the smaller tree under the larger one's root.
Kruskal MST	Sort all edges by weight. Add an edge if it doesn't create a cycle, using DSU.

TERM	EXPLANATION
Prim MST	Min-heap. Grow the MST outward from a source node, greedily adding the cheapest edge.
Floyd-Warshall	All-pairs shortest path in $O(V^3)$. $dp[i][j] = \min \text{ over all intermediate } k$.
Negative cycle	Bellman-Ford detects one when a distance still improves after $V-1$ relaxations.

CONCEPT QUESTIONS — answer in plain words

- Q1. Why does Dijkstra fail with negative edge weights? Give a concrete counterexample.
- Q2. Explain path compression in DSU find() - why does it matter for performance?
- Q3. What is the difference between Kruskal and Prim for MST? When to prefer each?
- Q4. What does Floyd-Warshall compute? When is $O(V^3)$ acceptable?
- Q5. How does Bellman-Ford detect a negative cycle?

CODING PROBLEMS — solve on LeetCode

- LC #743 Network Delay Time — Dijkstra from the source. Return the max distance among visited nodes.
- LC #684 Redundant Connection — DSU. The first edge that creates a cycle is the redundant connection.
- LC #787 Cheapest Flights Within K Stops — Bellman-Ford with at most k relaxation rounds.

BEFORE MOVING ON

MUST — do not move on without these

- Implement Dijkstra with a min-heap from memory.
- Implement DSU with path compression and union by rank.
- Solve Network Delay Time and Redundant Connection.

SOFT — good to have, not blocking

- Implement Kruskal's MST using DSU from scratch.

RESOURCE: Striver Advanced Graph Series (YouTube)

24

Segment Trees

Range queries and point updates both in $O(\log n)$. Better than prefix sum for updates.
1-1.5 weeks | 1-3 hrs/day | Difficulty: Hard

WHAT IS IT?

A segment tree is a binary tree where each node stores an aggregate (sum, min, or max) of a range. Both range query and point update run in $O(\log n)$. Prefix sum gives $O(1)$ query but $O(n)$ update - a segment tree balances both. Lazy propagation extends this to $O(\log n)$ range updates too.

KEY CONCEPTS

TERM	EXPLANATION
Tree structure	n leaves represent n elements. Total nodes needed are at most 4n.
Build $O(n)$	Recursive. Leaf: $tree[node] = arr[i]$. Internal node combines its two children.
Query $O(\log n)$	Node's range fully inside the query: return it. Fully outside: return identity.
Update $O(\log n)$	Update the leaf, then propagate up, recomputing every ancestor along the path.
Lazy propagation	Defer range updates. Store the pending update and apply it only when visited.
Node indexing	Root is index 1. Left child is $2*node$. Right child is $2*node+1$.
Identity element	Sum: 0. Min: INT_MAX. Max: INT_MIN - returned for non-overlapping ranges.

CONCEPT QUESTIONS — answer in plain words

- Q1.** Why does a Segment Tree need up to $4n$ nodes for n elements? Explain the bound.
- Q2.** How does the query work - what are the three cases at each node?
- Q3.** Explain lazy propagation - when do you apply the deferred update?
- Q4.** What is the identity element for min queries? Why must you return it for non-overlapping ranges?
- Q5.** Time complexity of building a Segment Tree - why is it $O(n)$ not $O(n \log n)$?

CODING PROBLEMS — solve on LeetCode

LC #307 Range Sum Query Mutable — Build the tree, then support point update and range sum query.

Practice range min — Build a min segment tree. Answer range minimum queries in $O(\log n)$.

LC #315 Count of Smaller Numbers After Self — Modified merge sort, or a BIT/Segment Tree over the value range.

BEFORE MOVING ON

MUST — do not move on without these

- Implement build, query, and update for a sum segment tree from scratch.
- Explain the three-case query logic - inside, outside, and partial overlap.
- Solve LC #307 Range Sum Query Mutable.

SOFT — good to have, not blocking

- Implement lazy propagation for range update queries.

RESOURCE: CP-Algorithms Segment Tree article + Striver (YouTube)

WHAT IS IT?

This extends 1D/2D DP to more complex state spaces. Bitmask DP encodes subsets as integers. DP on trees uses DFS post-order to combine children's results. Digit DP counts integers satisfying a property digit by digit. These patterns appear in hard competitive programming and top-company hard interview rounds.

KEY CONCEPTS

TERM	EXPLANATION
Bitmask DP	State includes a bitmask of which elements have been used so far.
TSP bitmask	$dp[mask][i]$ = minimum cost to visit all cities in mask, ending at city i .
DP on trees	DFS post-order. $dp[node]$ is computed from the already-computed dp of its children.
Rerooting	Computes $dp[node]$ for every possible root in two DFS passes instead of n separate ones.
Digit DP	Counts numbers in a range satisfying a property, digit by digit with a 'tight' flag.
Interval DP	$dp[i][j]$ is the answer for the subarray $[i..j]$, filled by increasing interval length.
Matrix chain	$dp[i][j]$ = minimum cost to multiply matrices i through j , split at every possible point.

CONCEPT QUESTIONS — answer in plain words

Q1. What does the bitmask represent in bitmask DP? How do you iterate subsets of a mask?

Q2. How does DP on trees differ from standard 1D/2D DP? What is the traversal order?

Q3. Explain the 'tight' constraint in digit DP - what breaks without it?

Q4. What is interval DP? Describe the order in which you fill the dp table.

Q5. Why is bitmask DP limited to $n \leq 20$ or so? What is the state space size?

CODING PROBLEMS — solve on LeetCode

LC #337 House Robber III — Tree DP. Each node returns a (rob_this, skip_this) pair.

LC #312 Burst Balloons — Interval DP. $dp[i][j]$ = max coins from bursting all balloons in $[i..j]$.

LC #1986 Minimum Number of Work Sessions — Bitmask DP over which tasks have been scheduled.

BEFORE MOVING ON

MUST — do not move on without these

- Have solid 1D and 2D DP complete before starting this topic.
- Implement House Robber III using tree DP - return a pair from each node.
- Understand bitmask iteration over subsets.

SOFT — good to have, not blocking

- Solve Burst Balloons - the hardest classic interval DP problem.

RESOURCE: Codeforces EDU + Striver Advanced DP (YouTube)

Learning Resources — Where to Study Each of the 25 Topics

Primary = watch this first. Backup = use if primary does not click. Striver: best overall. Aditya Verma: best for Backtracking/Sliding Window/DP/Heaps. CP-Algorithms: best for Phase 5.

TOPIC	PRIMARY — watch this first	BACKUP — if primary does not click
Phase 1 — Foundations		
01. Complexity Analysis	Striver — Time & Space Complexity (YouTube, ~45 min). Search: 'Striver time complexity'	Abdul Bari — Asymptotic Notations (YouTube). More mathematical, good second watch.
02. C++ STL Essentials	Striver — C++ STL Full Series (YouTube, ~3 hrs). Search: 'Striver C++ STL'	CodeWithHarry — C++ STL Tutorial (YouTube). Slower pace for absolute beginners.
03. Recursion	Apna College — Recursion playlist, first 5 videos (YouTube). Search: 'Apna College recursion'	Striver — Recursion Series (YouTube). More problems, pattern-based approach.
04. Arrays	Striver — A-Z Sheet Arrays section Easy→Medium→Hard (YouTube + takeuforward.org)	Apna College — Arrays playlist (YouTube). Gentler pace, more intuition building.
05. Strings	Apna College — Strings playlist (YouTube). Search: 'Apna College strings'	Striver — A-Z Sheet Strings section (YouTube). Problem-first approach.
06. Linked List	Apna College — Linked List full playlist (YouTube). Best beginner LL series available.	Striver — Linked List Series (YouTube). More interview-focused problem set.
07. Stack & Queue	Striver — Stack & Queue Series (YouTube). Covers all patterns including monotonic stack.	Apna College — Stack & Queue playlist (YouTube). Good for conceptual understanding.
08. Hashing & Hashmaps	Striver — Hashing Series (YouTube). All hashmap patterns covered in ~3 videos.	NeetCode — Arrays & Hashing section (neetcode.io). Short tight explanations.
09. Searching	Striver — Binary Search Series (YouTube, ~25 videos). Most complete free BS resource.	Apna College — Binary Search playlist (YouTube). Slower pace, beginner friendly.
10. Sorting Algorithms	Apna College — Sorting playlist (YouTube). Implements all algorithms from scratch.	Striver — Sorting videos in A-Z Sheet (YouTube). More interview context.
11. Bit Manipulation	NeetCode — Bit Manipulation playlist (YouTube, 6 videos, one sitting ~1.5 hrs).	Striver — Bit Manipulation Series (YouTube). More problems with detailed walkthroughs.
Phase 2 — Patterns		
12. Two Pointers	Striver — Two Pointer Series (YouTube). Covers opposite and same direction variants.	NeetCode — Two Pointers section (neetcode.io). Concise pattern-focused explanations.
13. Sliding Window	Striver — Sliding Window Series (YouTube). Fixed and variable window patterns.	Aditya Verma — Sliding Window playlist (YouTube). Best beginner SW series, very patient.
14. Binary Trees	Apna College — Binary Trees full playlist (YouTube). Best beginner tree series.	Striver — Binary Tree Series (YouTube, ~50 videos). Very detailed, interview-focused.

TOPIC	PRIMARY — watch this first	BACKUP — if primary does not click
15. Binary Search Tree	Striver — BST Series (YouTube). All BST operations and interview problems covered.	Apna College — BST playlist (YouTube). More conceptual, good for invariant understanding.
16. Heaps & Priority Queue	Striver — Heaps Series (YouTube). Heap internals and all top-K patterns.	Aditya Verma — Heap playlist (YouTube). Pattern-based, very clear for beginners.
17. Math & Number Theory	NeetCode — Math section (neetcode.io) + GFG 'Modular Arithmetic' article (must read).	Striver — Math videos in A-Z Sheet (YouTube). GCD, Sieve, fast exponentiation.
Phase 3 — More Patterns		
18. Graphs (BFS & DFS)	Striver — Graph Series (YouTube, ~50 videos). Most complete free graph resource available.	Apna College — Graph playlist (YouTube). Gentler intro — do this before Striver.
19. Backtracking	Aditya Verma — Backtracking playlist (YouTube). Best for beginners, draws decision trees.	Striver — Recursion & Backtracking Series (YouTube). Pattern-wise with decision trees.
20. Tries	Striver — Tries Series (YouTube). Covers implementation and all key interview problems.	NeetCode — Tries section (neetcode.io). Concise implementation walkthrough.
Phase 4 — Advanced Algorithms		
21. Dynamic Programming	Aditya Verma — DP playlist (YouTube, watch first). Then Striver DP Series (~56 videos in strict order).	LeetCode Explore — Dynamic Programming (interactive problems with hints).
22. Greedy Algorithms	Striver — Greedy Series (YouTube). All standard greedy patterns with proofs.	GFG — Greedy Algorithms articles (geeksforgeeks.org). Good exchange argument proofs.
Phase 5 — Advanced DS		
23. Advanced Graphs	Striver — Graph Series Part 2 (YouTube, Dijkstra onwards) + CP-Algorithms Graphs articles.	cp-algorithms.com → Graphs. Best written reference for Dijkstra, Bellman-Ford, DSU, MST.
24. Segment Trees	CP-Algorithms — Segment Tree article (cp-algorithms.com). Most detailed free resource.	Striver — Segment Tree video (YouTube). Good intro before reading CP-Algorithms.
25. Advanced DP	Codeforces EDU — DP section (codeforces.com/edu). Structured lessons with problems.	Striver — Advanced DP videos (YouTube). Bitmask, tree DP, interval DP covered.